

专业综合实践与训练

“网络游鱼”

九分钟讲解视频录制手册

逐字稿 · 镜头表 · 三机操作卡 · 测试证据

项目	多媒体展示系统——网络游鱼
讲解人	张方晋
目标时长	8 分 55 秒（硬上限 10 分钟）
录制形式	屏幕录制 + 旁白；跨屏段使用三机广角画面

版本：录制就绪版 · 2026-06-20

1. 录制前十分钟快速准备

目标：先把三机联动调通，再开始正式旁白。不要把排障过程录进成片。

三台电脑

设备	角色	启动命令
左侧电脑	展示节点	<code>venv/bin/python -m cyberfish --display-node</code>
中间电脑	管理员	<code>venv/bin/python -m cyberfish --admin</code>
右侧电脑	展示节点	<code>venv/bin/python -m cyberfish --display-node</code>

开始录制前检查

- 三台电脑连接同一局域网，系统防火墙允许 Python 接收 UDP；默认端口为 37777。
- 每台电脑使用独立 config.json。若项目文件来自同一次复制且已包含配置，可分别指定 `--config config-left.json`、`config-center.json`、`config-right.json`。
- 中间管理员画面显示“在线主机 2”，并记录左右电脑各自的 node_id，避免选错物理方向。
- 将中间电脑的左邻居和右邻居手动设置正确，等待约 2 秒让互逆关系同步。
- 为提高跨屏素材出现概率，将鱼数量提高到 16 至 20 条、速度提高到 1.5 至 2.0，并提前录制 60 至 90 秒后剪取完整跨屏片段。
- 关闭聊天通知、系统更新提示和包含隐私信息的窗口；三台电脑亮度和色温尽量一致。
- 提前打开本材料 assets 目录中的架构图、时序图、管理员界面和当前测试结果图片。

录制顺序

推荐先录素材、后录旁白：报告图与程序画面 → 三机广角跨屏 → 测试结果 → 按纯提词稿统一配音 → 剪辑到 8 分 30 秒至 9 分钟。

2. 九分钟镜头总表

讲解主线：问题 → 方案 → 架构 → 两个难点 → 三机演示 → 测试 → 不足。

时间	主题	画面	必须说清
0:00-0:35	开场：项目是什么	报告封面	姓名、题目、独立完成、跨屏目标
0:35-1:20	问题分析：需要解决什么	报告中的业务流程图或需求分析页	自动发现、鱼动画、跨屏唯一性
1:20-2:00	方案选择：为什么使用 Pygame	技术路线文字页，可叠加关键词字幕	比较方案后选择 Pygame、UDP、JSON
2:00-2:50	总体设计：分层架构与主循环	图 4-1 系统架构图	六层结构、非阻塞网络、60 FPS 主循环
2:50-3:45	难点一：屏幕拓扑	三台电脑左-中-右示意或拓扑状态区域	四方向邻居、互逆关系、自动+手动
3:45-4:45	难点二：可靠的跨屏移交	图 5-2 游鱼跨屏移交时序图	transfer、ACK、重试、去重、恢复
4:45-5:25	动画与控制台实现	管理员程序单机画面	伪 3D、摆尾、气泡、控制台
5:25-7:30	核心演示：三机联动	能同时拍到左、中、右三台电脑的广角画面	两节点在线、左右拓扑、跨屏、同步控制
7:30-8:15	测试：87 项全部通过	本材料附带的 test-results-current.png	unittest、87 项、OK、覆盖范围
8:15-8:55	总结、不足与改进	程序全景或报告总结页	完成内容、未实测指标、后续改进

剪辑红线：成片不得超过 10:00；三机联动画面至少保留 1 分钟。

3. 逐字讲解与镜头动作

使用方法：旁白只读蓝色“讲解词”；灰色“画面/操作”用于剪辑提示，不朗读。自然停顿即可，不要逐字念代码。

0:00-0:35 开场：项目是什么

画面	报告封面
操作	封面停留约 5 秒，再缓慢放大题目和姓名。

讲解词

老师好，我是计算机科学与技术专业的张方晋。本次专业综合实践的题目是《多媒体展示系统——网游游鱼》。项目由我独立完成，目标是在同一局域网的多台电脑之间，实现游鱼连续跨屏移动的多媒体展示效果。

0:35-1:20 问题分析：需要解决什么

画面	报告中的业务流程图或需求分析页
操作	用鼠标依次指向自动发现、游鱼动画、跨屏移交三个关键词。

讲解词

系统主要解决三个问题：第一，多台主机如何自动发现并确定上下左右关系；第二，如何生成具有随机方向、速度、大小、颜色和摆尾效果的游鱼动画；第三，鱼离开当前屏幕后，如何保持原有状态进入相邻屏幕，并尽量避免重复和丢失。除此之外，系统还需要提供水流音效、参数控制、联机状态显示和异常降级功能。

1:20-2:00 方案选择：为什么使用 Pygame

画面	技术路线文字页，可叠加关键词字幕
操作	依次显示 Python、Pygame、UDP、JSON 四个关键词。

讲解词

设计阶段比较了 JavaFX、Unity、OpenGL 和 Python Pygame 等方案。考虑到课程项目需要快速部署、普通电脑即可运行，并且功能需求允许二维或三维动画，我最终选择 Python 加

Pygame。画面采用二维绘制，通过深度缩放、阴影、旋转和层次排序形成伪三维效果；网络部分采用 UDP，配置使用 JSON 文件，不需要部署数据库或中心服务器。

2:00-2:50 总体设计：分层架构与主循环

画面	图 4-1 系统架构图
操作	从上到下指出展示层、应用层、模型/网络/拓扑层和支撑层。

讲解词

系统采用分层设计。界面展示层负责鱼群、背景、水波纹和控制台；应用协调层负责主循环和模块调度；模型层负责鱼的运动、边界检测和状态序列化；网络层负责 UDP 报文；拓扑层维护上下左右邻居；支撑层负责配置、音效和测试。主循环按照事件处理、网络轮询、鱼群更新、状态同步和画面渲染的顺序运行，目标帧率为每秒六十帧。

2:50-3:45 难点一：屏幕拓扑

画面	三台电脑左-中-右示意或拓扑状态区域
操作	突出 left、right、up、down 以及互逆关系。

讲解词

第一个实现难点是屏幕拓扑。系统把每台电脑看成一个节点，拓扑中保存左、右、上、下四个邻居。节点通过 DISCOVER、HEARTBEAT 和 TOPOLOGY_UPDATE 消息发现彼此并协商关系。自动协商无法准确反映实际摆放时，管理员可以在控制台选择在线主机并手动指定方向。相邻关系要求互逆，例如本机右侧是节点 B，那么节点 B 的左侧就应当是本机。

3:45-4:45 难点二：可靠的跨屏移交

画面	图 5-2 游鱼跨屏移交时序图
操作	沿时序图依次指向 fish_transfer、transfer_ack、重试和恢复。

讲解词

第二个难点是 UDP 不保证可靠送达。鱼越过开放边界后，发送端把鱼的编号、位置、速度、大小、颜色和深度等状态封装为 fish_transfer 消息。接收端在对应边缘重建鱼，并返回 transfer_ack。发送端只有确认成功后才完成所有权切换；如果没有收到确认，会短间隔重试，

超时后将鱼恢复到本机边缘。接收端还会根据 transfer_id 去重，从而降低鱼重复或永久丢失的概率。

4:45-5:25 动画与控制台实现

画面	管理员程序单机画面
操作	展示鱼的深浅层次、阴影、摆尾、气泡以及左上角控制台。

讲解词

动画方面，每条鱼都有独立编号、速度、颜色、大小、深度和动画状态。运动结合随机目标以及简单的分离、对齐和聚合趋势。渲染器绘制鱼身、尾鳍、阴影、气泡和水波纹。不同深度的鱼采用不同缩放比例，因此画面具有一定空间层次。左上角控制台可以调整鱼数量、速度、暂停、重置、音效、网络 and 全屏状态。

5:25-7:30 核心演示：三机联动

画面	能同时拍到左、中、右三台电脑的广角画面
操作	先拍在线主机数，再设置左右邻居；保留一次完整跨屏；随后演示暂停/继续、鱼数量+、速度+。

讲解词

下面进行实际演示。中间电脑作为管理员，左右电脑作为展示节点。可以看到系统已经发现两台在线主机。我先选择左侧节点并设置为左邻居，再把另一台设置为右邻居。现在鱼从中间屏幕左边游出后，会从左侧电脑的右边进入；向右游出时则进入右侧电脑。鱼的方向、速度、大小和颜色保持不变。接下来演示暂停、继续、增加鱼数量和调节速度。这些命令由管理员发送，展示节点同步执行，并返回处理结果。

7:30-8:15 测试：87 项全部通过

画面	本材料附带的 test-results-current.png
操作	确保终端中的 Ran 87 tests 和 OK 清晰可见。

讲解词

项目使用 unittest 编写了配置、鱼模型、网络协议、拓扑协商、控制台、应用逻辑和无窗口冒烟测试。我刚刚实际执行了完整测试，共运行八十七项，结果全部通过。其中包括本机回环环境下的节点发现、鱼移交、ACK 确认和超时恢复测试。

8:15-8:55 总结、不足与改进

画面	程序全景或报告总结页
操作	回到三机运行全景，最后停留 2 秒再结束录制。

讲解词

本项目完成了从需求分析、架构设计到编码和测试的完整软件工程过程，实现了伪三维游鱼动画、局域网发现、拓扑协调、跨屏移交、管理员控制和水流音效。当前不足是跨屏延迟小于一百毫秒以及连续运行三十分钟仍属于现场验收目标，还需要在更多真实设备上持续测量。后续可以增加拖拽式屏幕校准、更精细的鱼模型和网络诊断功能。我的讲解完毕，谢谢老师。

4. 三机演示操作卡

这两分钟只证明四件事：发现在线节点、拓扑正确、鱼真实跨屏、管理员命令同步。

步骤	目的	操作	录制判据
1	确认组网	管理员状态显示在线主机数为2；左右展示节点显示当前管理员已连接。	画面必须拍清在线数量。
2	识别节点	对照左右电脑显示的 node_id，在管理员在线主机按钮中找到对应节点。	不要根据按钮顺序猜物理位置。
3	设置左邻居	选中左侧电脑 node_id，点击“左”。	等待拓扑文字更新。
4	设置右邻居	选中右侧电脑 node_id，点击“右”。	等待约 2 秒完成互逆同步。
5	捕捉跨屏	保持三机同时入镜，录到鱼从中间完整游出并从相邻屏幕对应边缘进入。	完整过程不要切镜头。
6	同步控制	依次点击暂停、继续、鱼数量+、速度+，观察展示节点同步变化及 ACK 状态。	至少保留两项控制。

现场不要做的操作

- 不要在核心演示中点击“网络关”，否则展示节点会断开，恢复过程会浪费录制时间。
- 不要等待随机鱼很久才开始说话；先录足素材，再把最完整的一次跨屏剪入旁白位置。
- 不要只拍管理员屏幕；核心证据必须同时看到鱼离开一台电脑并进入另一台电脑。

常见故障快速处理

现象	处理
发现不到节点	确认同一局域网、UDP 37777 未被防火墙拦截；临时加 --debug-net 查看收发统计。
节点编号冲突	为每台电脑指定不同的 --config 文件，确保首次启动分别生成 node_id。
鱼不跨屏	确认网络为开、邻居在线且左右方向正确；提高鱼数量和速度，连续录制 60 至 90 秒。
没有声音	检查系统输出设备；音频失败会自动降级，不影响跨屏核心演示。

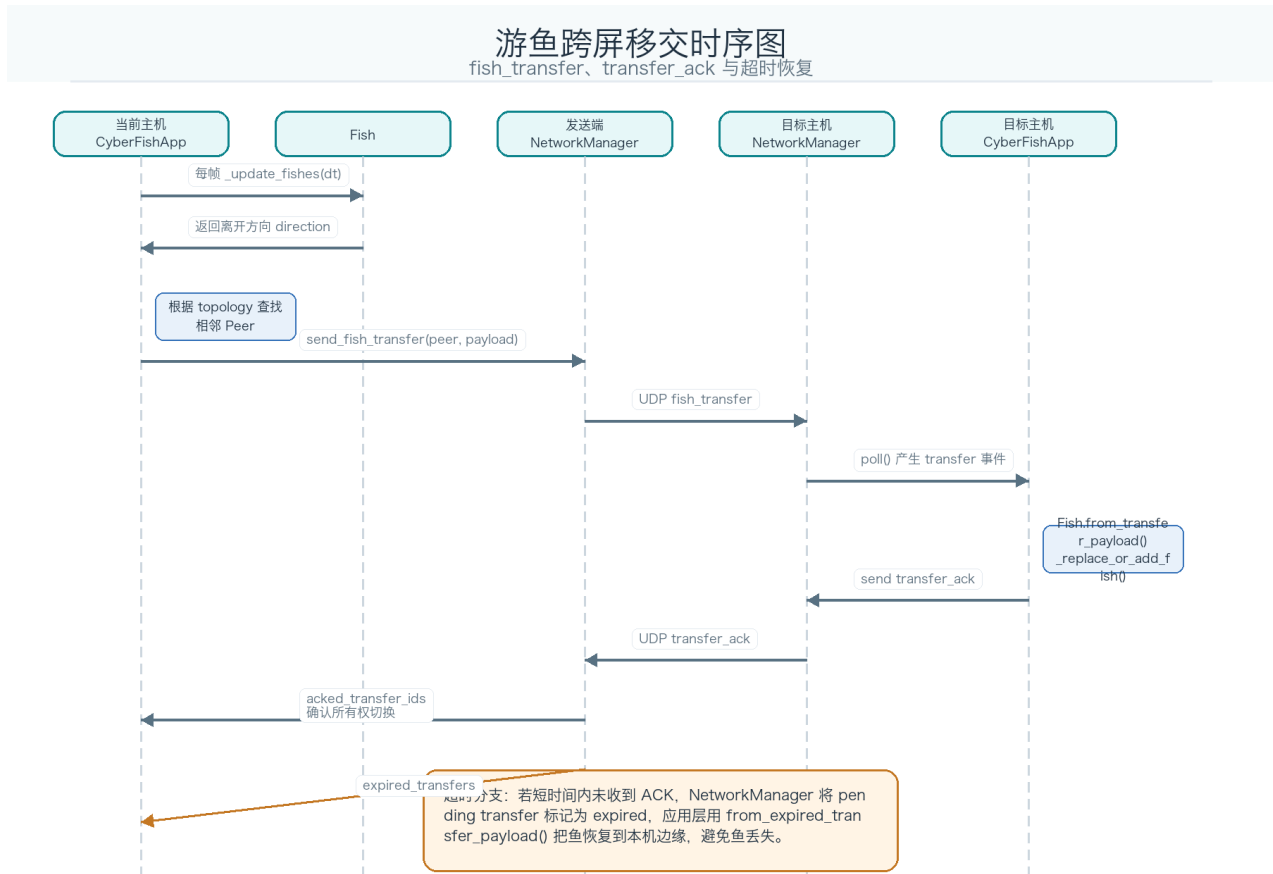
5. 录屏素材与测试证据

当前代码已重新执行完整测试：Ran 87 tests, 结果 OK。

系统架构图：用于 2:00-2:50 分层架构讲解。



跨屏移交时序图：用于 3:45-4:45 讲解 transfer 与 ACK。



管理员界面：用于说明控制台与状态显示。



当前测试结果 (建议直接作为 7:30 画面)

```
$ venv/bin/python -m unittest discover -v

... configuration / fish / network / topology / UI / smoke ...

-----
Ran 87 tests in 1.342s

OK

Verified locally: 2026-06-20 17:55:06
```

完整原始输出：assets/test-results-current.txt

6. 最终验收清单

完成	验收项
☐	总时长不超过 10:00，推荐 8:30 至 9:00。
☐	开场包含姓名、题目、个人贡献和项目目标。
☐	明确说明为什么选择 Python + Pygame、UDP 和 JSON。
☐	讲清屏幕拓扑与 UDP 可靠移交两个难点。
☐	三机画面能看到 2 台在线节点、左右拓扑和至少一次完整跨屏。
☐	至少演示两项管理员同步控制，并能看到展示节点变化。
☐	测试画面清晰显示 Ran 87 tests 和 OK。
☐	结尾如实说明 100ms 延迟与 30 分钟稳定性仍需真实现场测量。
☐	字幕、桌面和终端中不包含密码、聊天内容、IP 隐私或无关通知。
最终口径：已实现并通过自动化测试；性能指标不编造，留给真实三机现场验收。	